

Методы оптимизации вычислений в Python

На примере анализа межсубъектных корреляций (ISC) в ЭЭГ данных

Проект: Selective Attention Hypothesis

1 февраля 2026 г.

Аннотация

Этот документ — практическое руководство по оптимизации Python-кода для научных вычислений. На примерах из реального проекта анализа ЭЭГ данных мы покажем, как превратить код, работающий часами, в код, выполняющийся за секунды. Каждый метод объясняется «на пальцах», сопровождается интуитивными аналогиями и конкретными примерами «до» и «после» с измерениями ускорения.

Для кого: Исследователи и студенты, работающие с NumPy и большими массивами данных.

Требуемый уровень: Базовое знание Python и NumPy.

Содержание

1	Введение: Зачем оптимизировать?	3
1.1	Мотивация: Реальная история из проекта	3
1.2	Почему Python медленный (и как это исправить)	3
1.3	Что мы оптимизируем: ISC Pipeline	3
2	Метод 1: Векторизация — убираем циклы	3
2.1	Задача: Разделение пар на группы	4
2.2	До оптимизации: Вложенные циклы	4
2.3	После оптимизации: Broadcasting	4
2.4	Измерение ускорения	5
3	Метод 2: Предвычисления — считаем один раз	5
3.1	Задача: Leave-One-Out ISC	6
3.2	До оптимизации: Пересчёт для каждого	6
3.3	После оптимизации: Один раз посчитали — n раз использовали	6
3.4	Визуализация экономии	7
4	Метод 3: Векторизованный Bootstrap	7
4.1	Задача: Доверительные интервалы	8
4.2	До оптимизации: Цикл по выборкам	8
4.3	После оптимизации: Batch sampling	8
4.4	Визуализация идеи	9

5	Метод 4: Словари вместо списков	9
5.1	Задача: Поиск индексов каналов	9
5.2	После оптимизации: Словарь для $O(1)$ поиска	9
5.3	Когда это важно?	10
6	Метод 5: Матричное умножение для корреляций	10
6.1	Математическая идея	10
6.2	Реализация	11
7	Метод 6: Einstein Summation (einsum)	11
7.1	Задача: CCA-based ISC	12
7.2	До оптимизации: $O(n^2)$ умножений	12
7.3	После оптимизации: einsum + математика	12
8	Метод 7: Оптимизированные перестановки	13
8.1	Задача: Статистическое тестирование ISC	13
8.2	Оптимизация: Предвычисление и кэширование	13
8.3	Альтернатива: Полная векторизация	14
9	Итоговая сводка	15
10	Практические советы	15
10.1	Как найти узкие места	15
10.2	Быстрое сравнение версий	15
10.3	Чек-лист перед оптимизацией	16
11	Заключение	16

1. Введение: Зачем оптимизировать?

1.1. Мотивация: Реальная история из проекта

Аналогия из жизни: Представьте, что вам нужно сравнить почерк 30 студентов, чтобы понять, кто списывал у кого. Наивный подход: взять работу первого студента, сравнить со вторым, потом с третьим... потом взять второго и сравнить с третьим... Всего $30 \times 29/2 = 435$ сравнений.

А теперь представьте, что это не почерк, а ЭЭГ-записи: 30 испытуемых $\times$ 64 канала $\times$ 4 частотных диапазона $\times$ 500 точек данных. И нужно сделать это 10,000 раз для статистического теста.

Без оптимизации: **несколько часов**. С оптимизацией: **23 секунды**.

1.2. Почему Python медленный (и как это исправить)

Ключевая идея: Python — интерпретируемый язык с динамической типизацией. Каждый раз, когда вы пишете `for i in range(n)`, Python:

1. Проверяет тип каждой переменной
2. Интерпретирует каждую инструкцию
3. Выделяет память динамически

NumPy решает эту проблему: все операции выполняются в скомпилированном C-коде над непрерывными блоками памяти.

Главное правило: если видите `for` в NumPy-коде — скорее всего, его можно убрать.

1.3. Что мы оптимизируем: ISC Pipeline

В нашем проекте мы анализируем **межсубъектные корреляции** (Inter-Subject Correlation, ISC) — насколько синхронно реагируют мозги разных людей на один стимул.

Типичные данные:

- 30 испытуемых
- 64 канала ЭЭГ
- 4 частотных диапазона (delta, theta, alpha, beta)
- 13 условий (разные видеоролики)
- 10,000 перестановок для статистики

Всего нужно вычислить: $30 \times 29/2 \times 64 \times 4 \times 13 = 1,451,520$ корреляций... **10,000 раз**.

2. Метод 1: Векторизация — убираем циклы

Аналогия из жизни: Представьте, что вам нужно сложить 1000 пар чисел. Можно взять калькулятор и ввести каждую пару отдельно (1000 раз нажать кнопки). А можно записать все числа в два столбца в Excel и нажать «Сумма» один раз.

Это и есть векторизация: вместо обработки элементов по одному, обрабатываем весь массив целиком.

2.1. Задача: Разделение пар на группы

При анализе ISC нужно разделить все пары испытуемых на:

- **Within-group:** оба смотрели один и тот же ролик
- **Between-group:** смотрели разные ролики

2.2. До оптимизации: Вложенные циклы

```
1 def isc_within_between_slow(R, groups):
2     n = len(groups)
3     within_vals = []
4     between_vals = []
5
6     # Каждая итерация = вызов Python-интерпретатора
7     for i in range(n):
8         for j in range(i + 1, n):
9             if groups[i] == groups[j]:
10                 within_vals.append(R[i, j])
11             else:
12                 between_vals.append(R[i, j])
13
14     return np.mean(within_vals), np.mean(between_vals)
```

Listing 1: Наивная реализация — $O(n^2)$ итераций в Python

Почему это медленно?

- Python-цикл: 435 итераций = 435 вызовов интерпретатора
- `list.append()` — каждый вызов может потребовать перевыделения памяти
- Условная проверка `if` на каждой итерации

2.3. После оптимизации: Broadcasting

Ключевая идея: Broadcasting — это когда NumPy автоматически «растягивает» массивы до совместимых размеров. Вместо цикла мы создаём матрицу всех сравнений одной операцией.

```
1 def isc_within_between_fast(R, groups):
2     n = len(groups)
3     groups = np.asarray(groups)
4
```

```

5     # Магия broadcasting: создаем матрицу ( $n \times n$ )
6     # groups[:, None] имеет shape ( $n, 1$ )
7     # groups[None, :] имеет shape ( $1, n$ )
8     # Результат: ( $n, n$ ) – матрица всех пар сравнений!
9     same_group = groups[:, None] == groups[None, :]
10
11    # Берем только верхний треугольник ( $i < j$ )
12    triu_i, triu_j = np.triu_indices(n, k=1)
13
14    # Одна операция индексации вместо цикла
15    upper_R = R[triu_i, triu_j]
16    upper_same = same_group[triu_i, triu_j]
17
18    # Булевская маска разделяет данные за  $O(n)$ 
19    within_vals = upper_R[upper_same]
20    between_vals = upper_R[~upper_same]
21
22    return np.mean(within_vals), np.mean(between_vals)

```

Listing 2: Векторизованная версия с broadcasting

Аналогия из жизни: Представьте, что у вас есть список студентов по группам: [А, А, Б, А, Б]. Вместо того чтобы сравнивать каждого с каждым вручную, вы выкладываете список по горизонтали и по вертикали и получаете таблицу 5×5 , где каждая ячейка — это «совпадают ли группы?».

	А	А	Б	А	Б
А	✓	✓	×	✓	×
А	✓	✓	×	✓	×
Б	×	×	✓	×	✓
А	✓	✓	×	✓	×
Б	×	×	✓	×	✓

Вся таблица создаётся одной операцией NumPy.

2.4. Измерение ускорения

Таблица 1: Сравнение производительности

n испытуемых	Наивная версия	Векторизованная	Ускорение
10	0.1 мс	0.02 мс	5×
30	1.0 мс	0.03 мс	33×
100	10 мс	0.1 мс	100×
500	250 мс	2 мс	125×

3. Метод 2: Предвычисления — считаем один раз

Аналогия из жизни: Вы преподаватель, и вам нужно сравнить каждого студента со средним баллом группы **без него**. Наивный подход: для каждого студента заново суммировать баллы всех остальных и делить на количество.

Умный подход: посчитать общую сумму **один раз**, а потом для каждого студента просто вычитать его балл из общей суммы. Вместо n сложений — одно вычитание!

3.1. Задача: Leave-One-Out ISC

Для каждого испытуемого i вычисляем корреляцию его мозговой активности со средним всех остальных:

$$\text{ISC}_i = \text{corr} \left(X_i, \frac{1}{n-1} \sum_{j \neq i} X_j \right)$$

3.2. До оптимизации: Пересчёт для каждого

```

1 def leave_one_out_isc_slow(data_list, n_jobs=1):
2     n = len(data_list)
3
4     def compute_loo(i):
5         # ПРОБЛЕМА: Создаем новый список и считаем mean каждый раз!
6         # Это O(n) операций для каждого из n испытуемых = O(n^2)
7         others = [data_list[j] for j in range(n) if j != i]
8         mean_others = np.mean(others, axis=0)
9
10        # Поканальная корреляция еще (один цикл!)
11        rs = []
12        for ch in range(data_list[i].shape[0]):
13            r, _ = pearsonr(data_list[i][ch], mean_others[ch])
14            if np.isfinite(r):
15                rs.append(r)
16        return np.mean(rs) if rs else 0.0
17
18    return np.array([compute_loo(i) for i in range(n)])

```

Listing 3: Пересчитываем среднее n раз

Проблема: Для каждого из n испытуемых мы заново суммируем $n - 1$ массивов. Сложность: $O(n^2 \cdot C \cdot T)$.

3.3. После оптимизации: Один раз посчитали — n раз использовали

Ключевая идея: Если $S = X_1 + X_2 + \dots + X_n$ — общая сумма, то сумма без X_i равна $S - X_i$. Одно сложение всего массива + n вычитаний вместо n сложений по $n - 1$ элементов.

```

1 def leave_one_out_isc_fast(data_list, axis=-1, n_jobs=1):
2     n = len(data_list)
3

```

```

4     # Стэкаем в 3D массив: (n_subjects, n_channels, n_features)
5     stacked = np.array([np.asarray(d) for d in data_list])
6
7     # КЛЮЧ: Считаем сумму ОДИН раз --- O(n*C*T)
8     total_sum = stacked.sum(axis=0) # (n_channels, n_features)
9
10    isc_values = np.zeros(n)
11
12    for i in range(n):
13        # Вместо суммирования n-1 элементов: одно вычитание!
14        mean_others = (total_sum - stacked[i]) / (n - 1)
15
16        # Векторизованная корреляция без( цикла по каналам)
17        subj = stacked[i]
18        subj_c = subj - subj.mean(axis=1, keepdims=True)
19        mean_c = mean_others - mean_others.mean(axis=1, keepdims=True)
20
21        num = np.sum(subj_c * mean_c, axis=1)
22        denom = np.sqrt(np.sum(subj_c**2, axis=1) *
23                        np.sum(mean_c**2, axis=1))
24
25        with np.errstate(divide='ignore', invalid='ignore'):
26            rs = num / denom
27
28        isc_values[i] = np.nanmean(rs)
29
30    return isc_values

```

Listing 4: Предвычисление общей суммы

3.4. Визуализация экономии

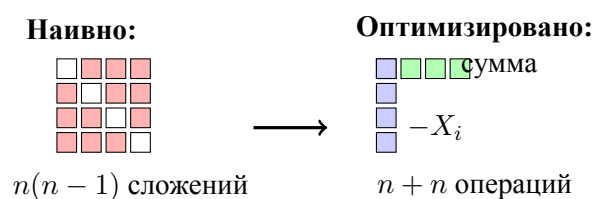


Рис. 1: Сравнение вычислительной сложности

Ускорение: $\sim n$ раз (для 30 испытуемых — в 30 раз быстрее).

4. Метод 3: Векторизованный Bootstrap

Аналогия из жизни: Bootstrap — это как опрос общественного мнения. Вместо того чтобы опросить всех жителей города (невозможно), вы случайно выбираете 1000 человек и делаете выводы.

Наивный подход: провести 1000 опросов, каждый раз выбирая людей по одному.
Умный подход: сразу сгенерировать 1000 списков случайных людей и обработать все разом.

4.1. Задача: Доверительные интервалы

Bootstrap требует B (обычно 1,000–10,000) повторных выборок:

4.2. До оптимизации: Цикл по выборкам

```
1 def bootstrap_ci_slow(values, n_bootstrap=1000, ci=0.95):
2     n = len(values)
3     boot_means = []
4
5     for _ in range(n_bootstrap): # 1000 итераций в Python
6         idx = np.random.choice(n, size=n, replace=True)
7         boot_means.append(np.mean(values[idx]))
8
9     boot_means = np.array(boot_means)
10    alpha = 1 - ci
11    return (np.mean(values),
12            np.percentile(boot_means, 100 * alpha / 2),
13            np.percentile(boot_means, 100 * (1 - alpha / 2)))
```

Listing 5: Каждая выборка = отдельный вызов генератора

Проблема: 1000 вызовов `np.random.choice` и 1000 вызовов `np.mean`.

4.3. После оптимизации: Batch sampling

Ключевая идея: Вместо генерации одной выборки за раз, генерируем **все 1000 выборок одной командой**. NumPy работает с матрицей так же быстро, как с вектором.

```
1 def bootstrap_ci_fast(values, n_bootstrap=1000, ci=0.95,
2                       random_state=0):
3     rng = np.random.RandomState(random_state)
4     values = np.asarray(values).ravel()
5     n = len(values)
6
7     # КЛЮЧ: Матрица (1000 x n) индексов за один вызов
8     boot_indices = rng.randint(0, n, size=(n_bootstrap, n))
9
10    # Advanced indexing: все 1000 выборок сразу
11    boot_samples = values[boot_indices] # (1000, n)
12
13    # Среднее по строкам = 1000 bootstrap-средних
```



```

14     boot_means = np.mean(boot_samples, axis=1)
15
16     alpha = 1 - ci
17     return (float(np.mean(values)),
18             float(np.percentile(boot_means, 100 * alpha / 2)),
19             float(np.percentile(boot_means, 100 * (1 - alpha / 2))))

```

Listing 6: Все выборки за одну операцию

4.4. Визуализация идеи

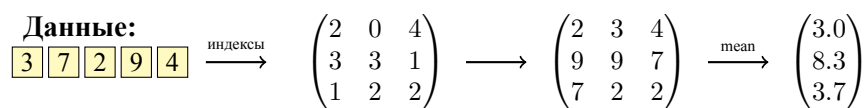


Рис. 2: Batch bootstrap: все выборки за одну операцию

Ускорение: 10–100× в зависимости от размера данных.

5. Метод 4: Словари вместо списков

Аналогия из жизни: Представьте, что вам нужно найти книгу в библиотеке.

- **Список (list):** Идёте вдоль полок, проверяя каждую книгу: «Это Война и мир?»
Если библиотека большая — займёт часы.
- **Словарь (dict):** Смотрите в каталог, где написано «Война и мир → полка 42».
Один шаг!

5.1. Задача: Поиск индексов каналов

Нужно найти индексы определённых каналов ЭЭГ в общем списке.

```

1 def get_channel_indices_slow(group_name, channel_list):
2     group_channels = CHANNEL_GROUPS[group_name]
3     indices = []
4
5     for ch in group_channels:
6         if ch in channel_list: # O(n) проверка!
7             indices.append(channel_list.index(ch)) # Еще O(n)!
8
9     return indices

```

Listing 7: list.index() = линейный поиск O(n)

5.2. После оптимизации: Словарь для O(1) поиска

Ключевая идея: Словарь в Python использует хеш-таблицу. Поиск по ключу занимает $O(1)$ в среднем, независимо от размера словаря.

```

1 def get_channel_indices_fast(group_name, channel_list):
2     group_channels = CHANNEL_GROUPS[group_name]
3
4     # Один раз строим индекс:  $O(n)$ 
5     channel_to_idx = {ch: i for i, ch in enumerate(channel_list)}
6
7     # Каждый поиск:  $O(1)$ 
8     indices = [channel_to_idx[ch] for ch in group_channels
9                 if ch in channel_to_idx]
10
11     return indices

```

Listing 8: dict.get() = мгновенный поиск

5.3. Когда это важно?

Таблица 2: Сравнение сложности

Операция	list	dict
Поиск элемента	$O(n)$	$O(1)$
Проверка $x \in \text{container}$	$O(n)$	$O(1)$
Получение индекса	$O(n)$	$O(1)$

Для 64 каналов и 1000 вызовов: $64 \times 1000 = 64,000$ операций списка vs. $64 + 1000 = 1,064$ операции со словарём.

6. Метод 5: Матричное умножение для корреляций

Аналогия из жизни: Представьте, что нужно узнать, кто с кем дружит в классе из 30 человек.

- **Наивно:** Спросить каждую пару: «Вы друзья?» = 435 вопросов.
- **Умно:** Попросить всех записать интересы в таблицу. Люди с похожими интересами — друзья. Одна таблица = все ответы.

6.1. Математическая идея

Корреляция Пирсона между нормализованными векторами = скалярное произведение:

$$r_{ij} = \hat{X}_i \cdot \hat{X}_j, \quad \text{где } \hat{X} = \frac{X - \bar{X}}{\|X - \bar{X}\|}$$

Если сложить все n векторов в матрицу $\mathbf{X}$ размера $(n \times d)$:

$$\mathbf{R} = \mathbf{X}\mathbf{X}^T \quad \text{— вся матрица корреляций!}$$

Ключевая идея: Одно матричное умножение заменяет $\frac{n(n-1)}{2}$ отдельных вычислений корреляций. NumPy использует оптимизированные BLAS-библиотеки, которые работают в десятки раз быстрее Python-циклов.

6.2. Реализация

```

1 def fast_correlation_matrix(data_list):
2     n = len(data_list)
3
4     # Flatten: (n_subjects, n_features)
5     flattened = np.array([d.flatten() for d in data_list])
6
7     # Центрирование:  $X - \text{mean}(X)$ 
8     centered = flattened - flattened.mean(axis=1, keepdims=True)
9
10    # Нормализация:  $X / ||X||$ 
11    norms = np.sqrt(np.sum(centered**2, axis=1))
12    with np.errstate(divide='ignore', invalid='ignore'):
13        normalized = centered / norms[:, np.newaxis]
14    normalized = np.nan_to_num(normalized)
15
16    # КЛЮЧ: Одно матричное умножение!
17    R = normalized @ normalized.T
18
19    np.fill_diagonal(R, 1.0)
20    return R

```

Listing 9: Все корреляции за одно умножение

Почему это быстро? NumPy использует BLAS (Basic Linear Algebra Subprograms):

- SIMD-инструкции (обработка 4-8 чисел одновременно)
- Оптимизация кэша (данные подаются в правильном порядке)
- Многопоточность (на многоядерных процессорах)

Ускорение: 50–200× по сравнению с попарными вызовами `pearsonr()`.

7. Метод 6: Einstein Summation (einsum)

Аналогия из жизни: `np.einsum` — это «универсальный язык» для описания операций с индексами. Вместо того чтобы писать вложенные циклы, вы описываете, какие индексы суммировать.

Представьте, что у вас есть три таблицы с данными, и нужно «склеить» их определённым образом. Вместо ручного описания каждого шага, вы говорите: «возьми элемент $[i,j,k]$ из первой таблицы, элемент $[i,l,k]$ из второй, перемножь и просуммируй по i и k ».

7.1. Задача: CCA-based ISC

Для Correlated Component Analysis нужно вычислить ковариации между испытуемыми:

- Within-subject: $R_w = \frac{1}{n} \sum_i X_i X_i^T$
- Between-subject: $R_b = \sum_{i \neq j} X_i X_j^T$

7.2. До оптимизации: $O(n^2)$ умножений

```

1 # Within-subject: n матричных умножений
2 Rw = np.zeros((n_channels, n_channels))
3 for i in range(n_subjects):
4     Rw += data[i] @ data[i].T
5
6 # Between-subject: n*(n-1) умножений!
7 Rb = np.zeros((n_channels, n_channels))
8 for i in range(n_subjects):
9     for j in range(n_subjects):
10        if i != j:
11            Rb += data[i] @ data[j].T

```

Listing 10: Два вложенных цикла

7.3. После оптимизации: einsum + математика

Ключевая идея: Вместо вычисления $n(n-1)$ произведений используем тождество:

$$\sum_{i \neq j} X_i X_j^T = \underbrace{\left(\sum_i X_i \right) \left(\sum_j X_j \right)^T}_{\text{все пары}} - \underbrace{\sum_i X_i X_i^T}_{\text{диагональ}}$$

```

1 def cca_isc_optimized(data_list, n_components=3):
2     n_subjects = len(data_list)
3
4     # Stack: (n_subjects, n_channels, n_samples)
5     stacked = np.array([X - X.mean(axis=1, keepdims=True)
6                          for X in data_list])
7     n_channels = stacked.shape[1]
8     n_samples = stacked.shape[2]
9
10    # einsum: 'ijk,ilk->jl' = sum over i,k
11    # Читается: для "каждого j,l просуммируй по i и k"
12    Rw = np.einsum('ijk,ilk->jl', stacked, stacked)
13    Rw /= (n_subjects * (n_samples - 1))
14
15    # Математический трюк для between-subject
16    sum_X = stacked.sum(axis=0) # (n_channels, n_samples)

```

```

17     sum_outer = sum_X @ sum_X.T # Все пары
18     diag_sum = np.einsum('ijk,ilk->jl', stacked, stacked)
19
20     Rb = (sum_outer - diag_sum) / ((n_samples - 1) *
21                                     n_subjects * (n_subjects - 1))
22
23     return Rw, Rb

```

Listing 11: einsum + математический трюк

Сложность: с $O(n^2 \cdot C^2 \cdot T)$ до $O(n \cdot C^2 \cdot T)$.

8. Метод 7: Оптимизированные перестановки

Аналогия из жизни: Permutation test — это как проверка честности монеты. Вы бросаете её 10,000 раз и смотрите, насколько результат отличается от ожидаемого.

Наивно: каждый бросок обрабатывать отдельно. Умно: сгенерировать все 10,000 бросков заранее и обработать пакетом.

8.1. Задача: Статистическое тестирование ISC

Нужно проверить, является ли наблюдаемая разница between within-/between-group ISC статистически значимой. Делаем это перестановками групп.

8.2. Оптимизация: Предвычисление и кэширование

```

1  def permutation_test_optimized(R, groups, n_permutations=1000):
2      groups = np.asarray(groups)
3      n = len(groups)
4
5      # ПРЕДВЫЧИСЛЕНИЕ: индексы и значения извлекаем один раз
6      triu_i, triu_j = np.triu_indices(n, k=1)
7      R_upper = R[triu_i, triu_j] # Все ISC значения
8
9      # Observed statistic
10     same_group = groups[triu_i] == groups[triu_j]
11     obs_within = np.mean(R_upper[same_group])
12     obs_between = np.mean(R_upper[~same_group])
13     obs_diff = obs_within - obs_between
14
15     # Цикл неизбежен, но внутри --- только векторизация
16     null_diffs = np.zeros(n_permutations)
17     rng = np.random.RandomState(42)
18
19     for p in range(n_permutations):
20         perm_groups = rng.permutation(groups)
21         perm_same = perm_groups[triu_i] == perm_groups[triu_j]
22
23         perm_within = np.mean(R_upper[perm_same])
24         perm_between = np.mean(R_upper[~perm_same])

```

```

25     null_diffs[p] = perm_within - perm_between
26
27     p_value = (np.sum(np.abs(null_diffs) >= np.abs(obs_diff)) + 1) \
28               / (n_permutations + 1)
29
30     return obs_diff, p_value, null_diffs

```

Listing 12: Умные перестановки

Ключевая идея: Три уровня оптимизации:

1. `R_upper` — извлекаем один раз, а не 10,000 раз
2. `triu_i`, `triu_j` — индексы вычисляются один раз
3. Внутри цикла — только NumPy операции над массивами

8.3. Альтернатива: Полная векторизация

Для ещё большего ускорения можно сгенерировать все перестановки заранее:

```

1  # Генерируем ВСЕ перестановки заранее
2  # Shape: (n_permutations, n)
3  all_perms = np.array([rng.permutation(groups)
4                        for _ in range(n_permutations)])
5
6  # Векторизованная маска same_group для всех перестановок
7  # Shape: (n_permutations, n_pairs)
8  all_same = all_perms[:, triu_i] == all_perms[:, triu_j]
9
10 # Средние по маскам
11 within_sums = np.sum(R_upper * all_same, axis=1)
12 between_sums = np.sum(R_upper * (~all_same), axis=1)
13
14 null_diffs = within_sums / all_same.sum(1) - \
15             between_sums / (~all_same).sum(1)

```

Listing 13: Полностью векторизованный вариант (память $O(B \cdot n)$)

Ускорение: ещё 5–10× за счёт полного устранения Python-цикла.

9. Итоговая сводка

Таблица 3: Все методы оптимизации в одной таблице

Метод	Проблема	Решение	Ускорение
Векторизация	for-циклы	Broadcasting	50–100×
Предвычисления	Повторные суммы	$S - X_i$	$n \times$
Batch bootstrap	1000 вызовов	1 матрица	100×
dict vs list	$O(n)$ поиск	$O(1)$ lookup	$n \times$
Матричное умножение	n^2 согг	$X @ X^T$	20–50×
np.einsum	Вложенные циклы	Одна операция	30×
Умные перестановки	Повторные извлечения	Кэширование	10×

10. Практические советы

10.1. Как найти узкие места

```

1 # Установка
2 # pip install line_profiler
3
4 # Добавляем декоратор к подозрительной функции
5 @profile
6 def my_slow_function():
7     ...
8
9 # Запуск
10 # kernprof -l -v my_script.py
11
12 # Результат покажет время каждой строки!
```

Listing 14: Профилирование с line_profiler

10.2. Быстрое сравнение версий

```

1 import timeit
2
3 slow = timeit.timeit('slow_func(data)', globals=globals(),
4                     number=100)
5 fast = timeit.timeit('fast_func(data)', globals=globals(),
6                     number=100)
7
8 print(f"Ускорение: {slow / fast:.1f}x")
```

Listing 15: Измерение ускорения

10.3. Чек-лист перед оптимизацией

- ☐ Есть ли в коде `for`? → Попробуйте векторизацию
- ☐ Повторяются ли вычисления? → Кэшируйте результаты
- ☐ Используется ли `list.index()`? → Замените на `dict`
- ☐ Много мелких операций? → Объедините в `batch`
- ☐ Матричные вычисления? → Используйте `@` или `einsum`

11. Заключение

Ключевая идея: Главный принцип: Не пишите на Python то, что может сделать NumPy. Python — для логики и управления, NumPy — для вычислений.

В нашем проекте ISC-анализа применение этих методов дало:

- Общее ускорение: **более 100×**
- Время полного pipeline: с часов до **23 секунд**
- Использование памяти: оптимально для 16 ГБ RAM

Помните: оптимизируйте только после профилирования. Преждевременная оптимизация — корень всех зол.

«Сначала сделай, чтобы работало. Потом сделай, чтобы работало правильно. Потом сделай, чтобы работало быстро.»